# Assignment 4

Name : Ravish Ranjan
Roll no. : 45
Course : MCA
Subject : Data Communication and Computer Network

## Question

Implement the Stop-and-Wait and Stop-and-Wait ARQ (Automatic Repeat reQuest) protocols. Simulate a communication channel with a frame loss probability of 70% and demonstrate the operation of both protocols. Verify and report the observed frame loss rate and compare the retransmission behaviour of the two protocols.

## Answer

```cpp
#include <random>
#include <utility>
#include <vector>

#define FRAME_LOSS_PROB 0.7
#define TOTAL_FRAMES 100

class Channel {
    private:
        std::mt19937 rand_dev;
        std::uniform_real_distribution<double> distrib;

    public:
        Channel(): rand_dev(std::random_device{}()), distrib(0.0,1.0) {}

        bool transmit(int frame_id) {
            bool is_success = distrib(rand_dev) >= FRAME_LOSS_PROB;
            if (!is_success) printf("%d,",frame_id);
            return is_success;
        }
};

std::vector<std::pair<std::string,double>> stop_and_wait(Channel* ch) {
    int total_count = 0, success_count = 0, failed_count = 0;

    printf("\nfailed attempt : ");

    for (int frame_id=1; frame_id<=TOTAL_FRAMES; frame_id++) {
        total_count++;

        if (ch->transmit(frame_id)) {
            success_count++;
        } else {
```

```
            failed_count++;
        }
    }

    printf("\n\n");

    std::vector<std::pair<std::string,double>> result;

    result.push_back({"total transmitted",total_count});
    result.push_back({"successful",success_count});
    result.push_back({"failed",failed_count});
    result.push_back({"re-transmitted",0});
    result.push_back({"loss-rate",(double)(failed_count*100) / total_count});
    result.push_back({"delivery-rate",(double)(success_count*100) /
total_count});

    return result;
}

std::vector<std::pair<std::string,double>> stop_and_wait_arq(Channel* ch) {
    int total_count = 0, success_count = 0, failed_count = 0, re_transmisit =
0;

    printf("\nfailed attempt : ");

    for (int frame_id=1; frame_id<=TOTAL_FRAMES; frame_id++) {
        int attempts = 0;

        while (true) {
            attempts++;
            total_count++;
            if (ch->transmit(frame_id)) {
                success_count++;
                break;
            } else {
                re_transmisit++;
            }
        }
    }

    printf("\n\n");

    std::vector<std::pair<std::string,double>> result;

    result.push_back({"total transmitted",total_count});
    result.push_back({"successful",success_count});
    result.push_back({"failed",failed_count});
    result.push_back({"re-transmitted",re_transmisit});
    result.push_back({"loss-rate",(double)(failed_count*100) / total_count});
    result.push_back({"delivery-rate",(double)(success_count*100) /
total_count});

    return result;
}
```

```
int main() {
    Channel* ch = new Channel();

    printf("---------------- Stop and Wait ----------------\n");

    auto snw_result = stop_and_wait(ch);

    for (const auto res:snw_result) {
        if (res.first == "loss-rate" || res.first == "delivery-rate")
            printf("%s : %.2f%\n",res.first.c_str(),res.second);
        else
            printf("%s : %.0f\n",res.first.c_str(),res.second);
    }

    printf("\n");

    printf("-------------- Stop and Wait ARQ --------------\n");

    auto snw_arq_result = stop_and_wait_arq(ch);

    for (const auto res:snw_arq_result) {
        if (res.first == "loss-rate" || res.first == "delivery-rate")
            printf("%s : %.2f%\n",res.first.c_str(),res.second);
        else
            printf("%s : %.0f\n",res.first.c_str(),res.second);
    }

    return 0;
}
```

## Output

```
ravish@ravishPC:../cn/prog ⎇ (main)> g++ -w ./a4.cpp -o a4.out
ravish@ravishPC:../cn/prog ⎇ (main)> ./a4.out
---------------- Stop and Wait ----------------

failed attempt :
1,2,3,4,5,6,8,9,10,11,12,13,14,15,16,17,23,24,25,28,29,30,31,32,33,35,36,38,3
9,40,41,43,44,45,47,48,49,50,51,52,57,58,61,62,63,64,65,66,67,69,70,71,72,74,
75,76,77,78,79,80,81,82,85,86,88,89,90,91,93,94,98,99,100,

total transmitted : 100
successful : 27
failed : 73
re-transmitted : 0
loss-rate : 73.00%
delivery-rate : 27.00%

-------------- Stop and Wait ARQ --------------

failed attempt :
```

```
1,1,2,5,6,6,7,7,7,7,7,7,7,10,10,10,10,10,10,10,10,12,12,12,12,12,13,13,13,13,
13,13,13,14,14,14,14,15,15,15,15,15,15,19,19,22,23,23,23,24,25,25,28,28,29,30
,30,30,32,32,32,32,34,34,35,35,35,36,36,38,38,38,38,38,38,38,39,40,40,40,42,4
2,43,44,44,44,46,47,47,47,47,47,47,47,47,47,47,47,47,47,51,51,52,53,53,54,
54,54,57,58,59,59,59,59,59,59,59,63,64,64,64,64,64,64,66,66,66,66,66,66,66,66
,66,66,66,67,69,70,70,70,70,70,70,70,71,73,73,73,74,75,75,79,79,80,80,80,81,8
1,81,83,86,86,86,86,86,86,86,86,87,89,91,91,91,91,91,91,91,94,94,96,96,97,97,
97,97,98,98,99,99,99,99,100,100,100,100,100,

total transmitted : 297
successful : 100
failed : 0
re-transmitted : 197
loss-rate : 0.00%
delivery-rate : 33.67%
ravish@ravishPC:../cn/prog ⎇ (main)>
```

## Observations

Stop and wait

| Result | Value |
|---|---|
| total transmitted | 100 |
| successful | 27 |
| failed | 73 |
| re-transmitted | 0 |
| loss-rate | 73.00% |
| delivery-rate | 27.00% |

Stop and wait ARQ

| Result | Value |
|---|---|
| total transmitted | 297 |
| successful | 100 |
| failed | 0 |
| re-transmitted | 197 |
| loss-rate | 0.00% |
| delivery-rate | 33.67% |

The loss rate in `stop and wait ARQ` is `0` cause it keeps transmissting the failed result if it fails transmittion attempt which has resulted in `197` almost twice the amount of re-transmission. In comparision `simple stop and wait` only tries each frame once and does no re-transmission